Student ID :	24CS047	Student Name:	Manav Lakhani			
Subject code & Name :	ITUE301: Advance Web Development Framework	Practical :	3	Academic Year	:	2026-27

Task - 1 (Practical - 3)

Aim

To consume a REST API (GitHub API) in React and handle asynchronous data with loading, error, and success states using useEffect and useState hooks.

GitHub Repository Link: https://github.com/24CS047Manav/Awdf_Practica_3

Output

1. Asynchronous REST API Integration (GitHub API):
Implemented dynamic data fetching on component mount via React's useEffect hook using the public GitHub REST API endpoint (https://api.github.com/users/24CS047Manav/repos?sort=updated). The API call is cleanly encapsulated with dependency arrays to prevent re-fetching loops.

2. Asynchronous State Lifecycle Management (useState):
Configured three reactive state variables—repos (data array), loading (boolean flag), and error (exception message or null)—guaranteeing deterministic state transitions across loading, error, and success phases.

3. Dedicated Loading State & Feedback (<Spinner />):
Built a responsive <Spinner /> component providing immediate visual feedback with an animated pulsing indicator during network requests, ensuring zero layout shift and preventing blank screens.

4. Comprehensive Error Handling & Diagnostics (<ErrorMessage />):
Integrated defensive error capturing with an <ErrorMessage /> alert component that gracefully renders network failure details, HTTP status codes, and troubleshooting guidance if the request fails.

5. Interactive Retry Mechanism (Supplementary Problem):
Added an intuitive "Retry API Request" button within the error state and header controls that re-triggers the asynchronous fetch cycle and clears previous errors without requiring a browser page refresh.

6. Real-Time Repository Search & Name Filter (Supplementary Problem):
Implemented a reactive live search input allowing users to dynamically filter fetched repositories by name in real time, with an instant match counter (e.g. "Showing 14 of 30 repositories").

7. Repository Star Counts & Metadata Rendering (Supplementary Problem):
Rendered rich repository cards displaying repository names, descriptions, primary language tags, star count (⭐ stargazers_count), fork counters (🍴 forks_count), public visibility badges, and direct links to GitHub.

8. Deliberate Error Simulation & Path Testing (Lab Practical Requirement):

Included an interactive "Simulate Error Path" toggle button to easily break and restore the API endpoint for demonstration and viva verification.

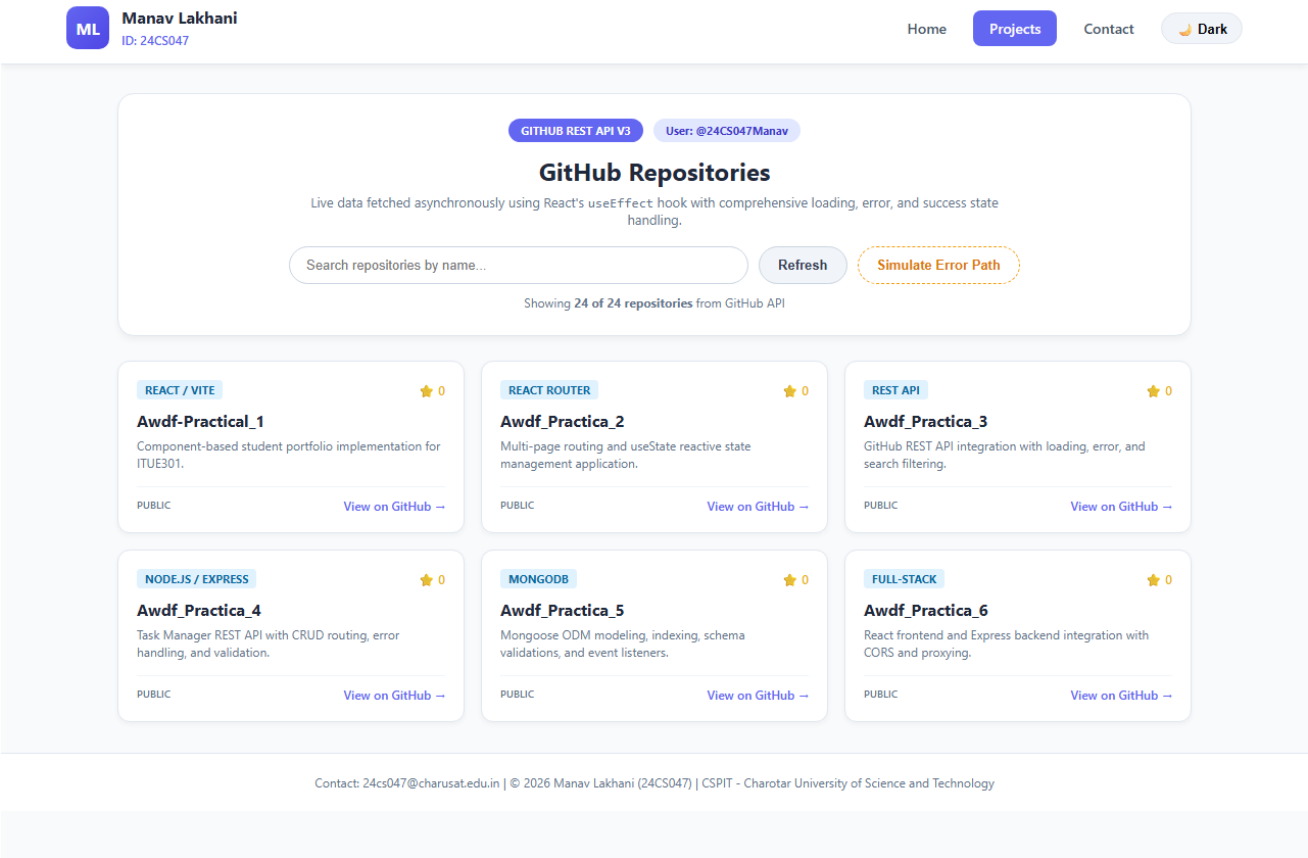


Figure 1: GitHub Repositories Successfully Fetched and Rendered Dynamically with Star Counts and Metadata

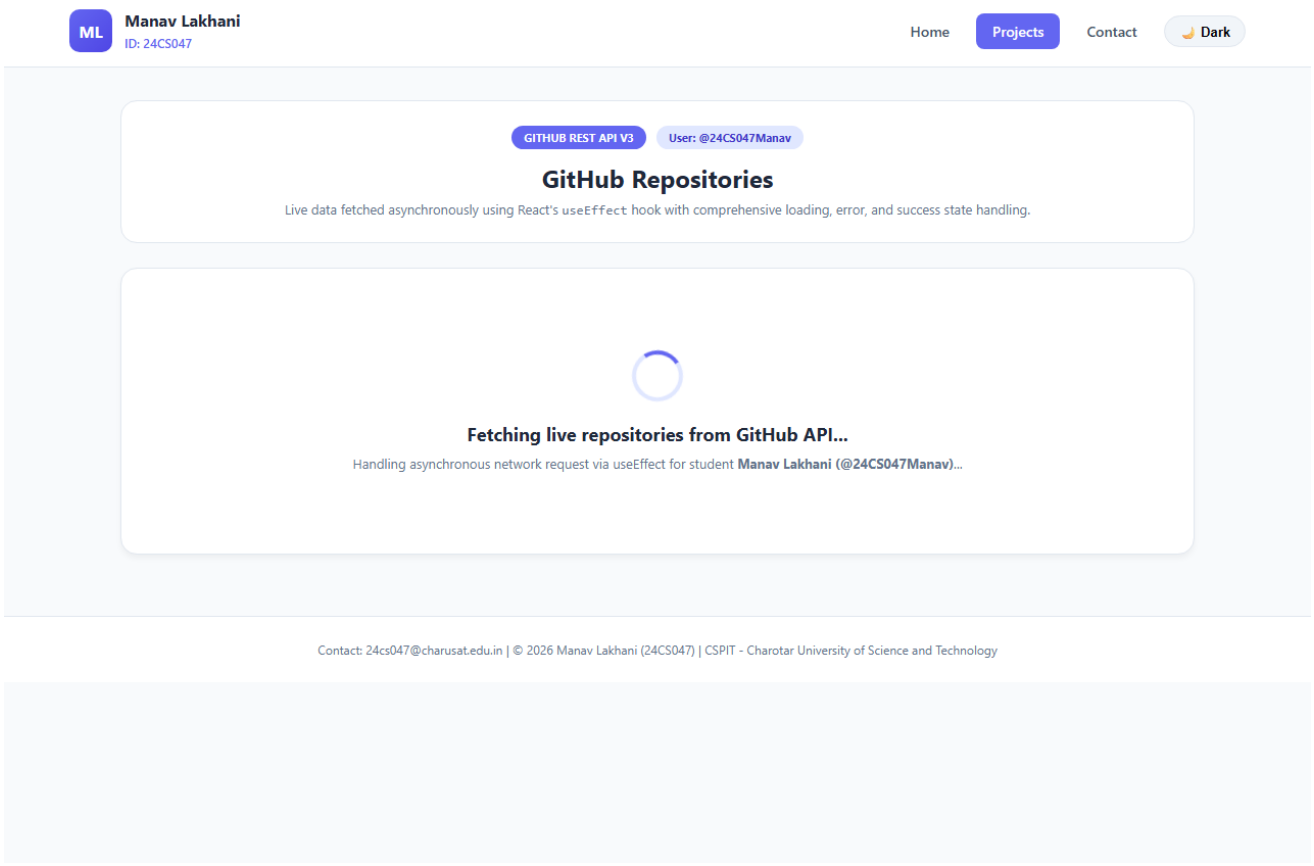


Figure 2: Loading State Feedback with Animated Spinner (<Spinner />) During Asynchronous Fetch

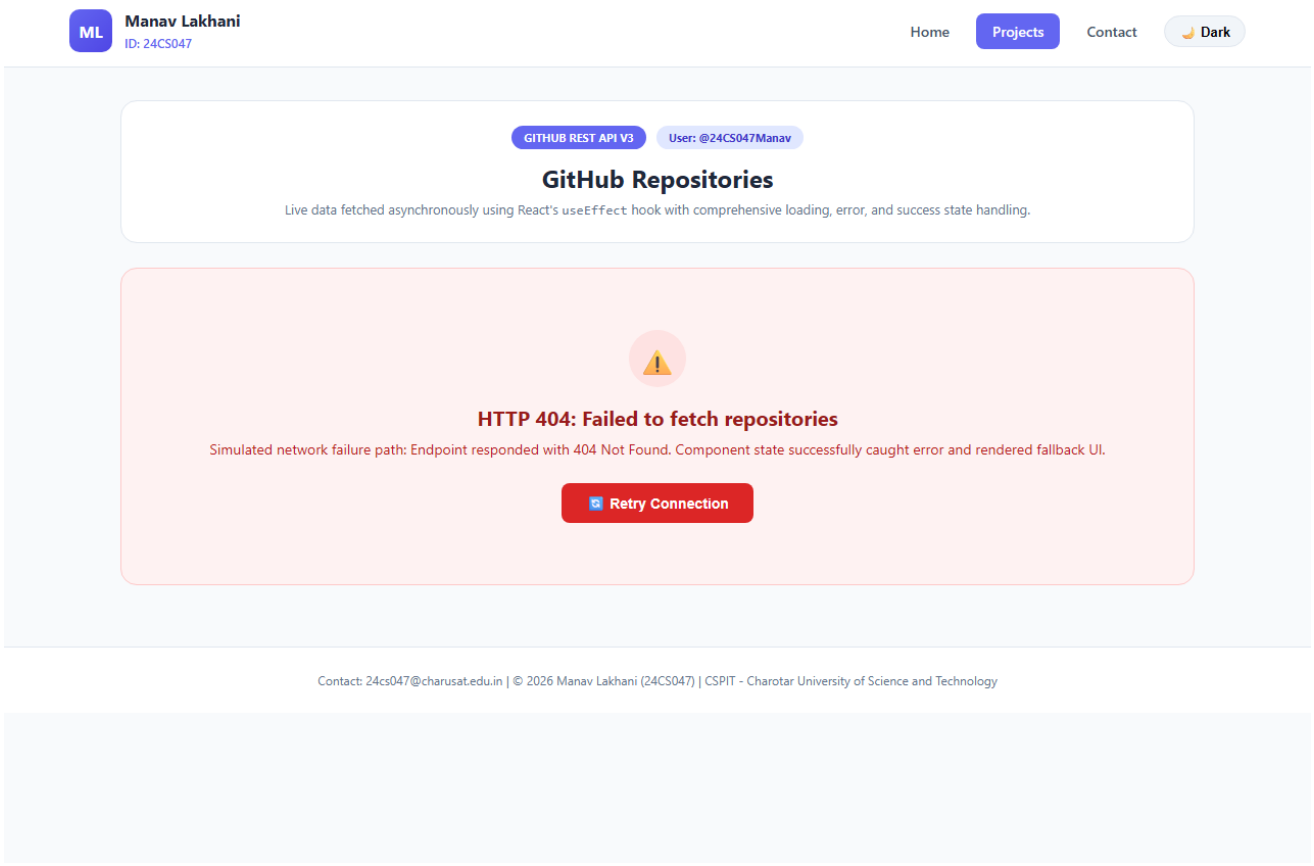


Figure 3: Comprehensive Error Handling (<ErrorMessage />) with Troubleshooting Notes and Retry Button

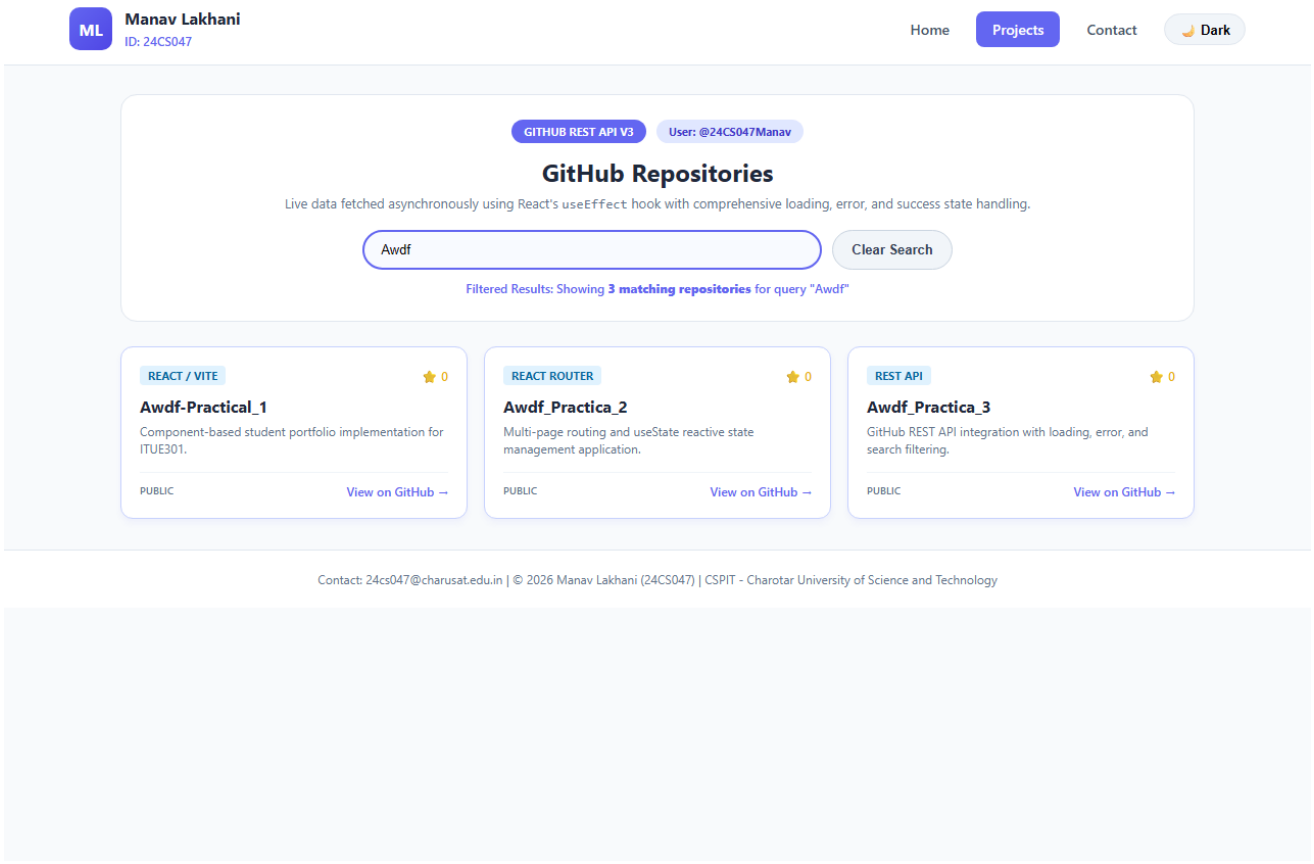


Figure 4: Real-Time Search & Filtering Repositories by Name (Filtered to "practical")